

Kurv

Kurv w Containers

Day 1 - Design

10-Mar-2026

Pythian

Agenda

- Overview
- Big Boxes and Responsibilities
- Design for your needs
- Shared Components
- Questions & Next Steps



Goals of the next 3 days

- A fully functional CI/CD pipeline.
 - Application being built from a repo
- A sample application deployed to a cloud environment.
 - Application once built will be deployed
- A roadmap for implementing a modern SDLC in your organization.
 - What could that roadmap look like to you
- All workshop materials, including presentations and code samples.
 - Repo shared

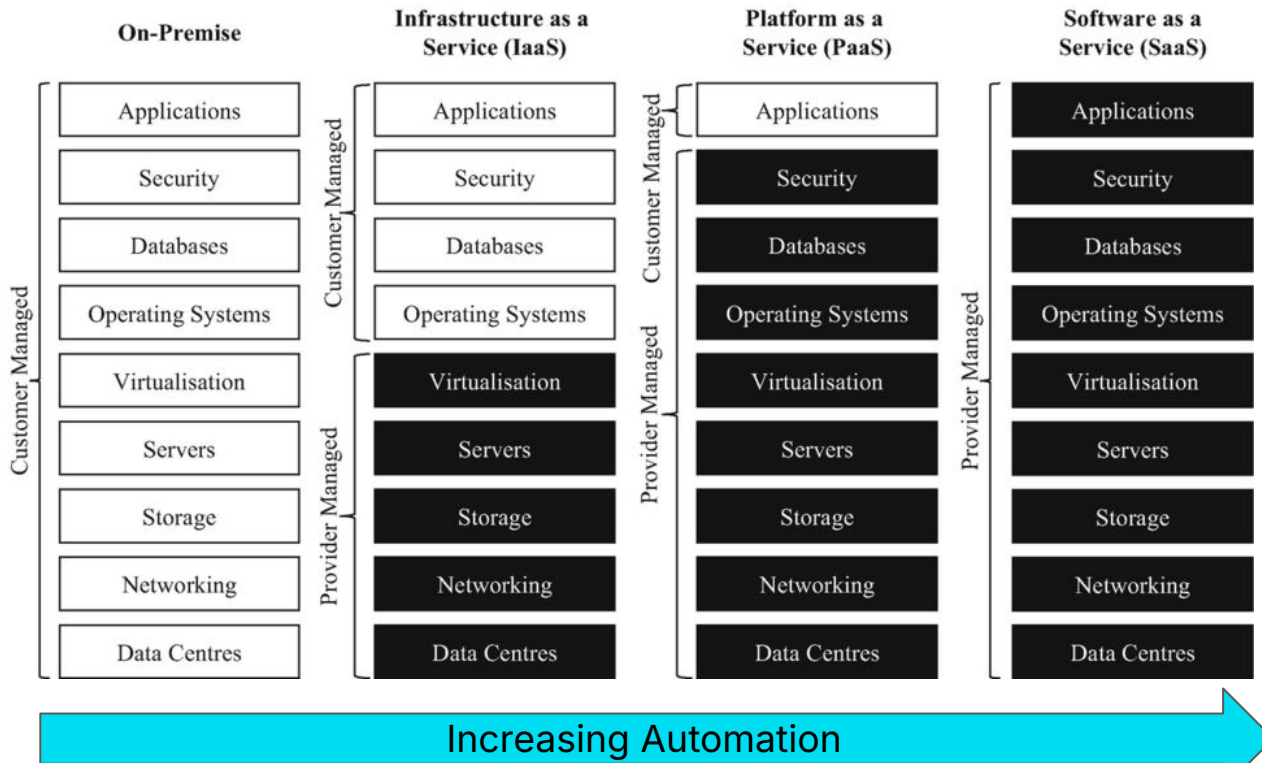
Technology



On Premise to the Cloud

What is in these buckets and how does this apply to my pipeline

This view of the diagram is basically where your view is and the increasing levels of automation. We want to take that approach to our development and deployment pipelines as much as possible.



3 Black Boxes and Boundaries

Base Infrastructure

There will be a base infrastructure, this will likely be “manually” deployed, configured etc, at least at the beginning.

This will be as basic as your AWS Account Tenant, all the way to centralized components that may run everything else.

DevOps

The infrastructure more directly associated to the applications that you are running for your application.

This will definitely have “environments” and some of these environments will be controlled differently (manual, fully automated, scheduled automated, approval requirements).

Development

The application that is being developed/maintained/deployed. What are the components that is needs, how are they controlled?

To development all environments need to look “logically” the same, how is this achieved?

3 Black Boxes and Boundaries

Base Infrastructure

What are the base items required and what are those boundaries. This will be driven by the organization needs, compliance needs, application needs.

- AWS
- Organizations Setup
- Regions Enabled
 - Production
 - non-Prod
 - DR
- Base Infrastructure (shared)
 - Networking/VPN
 - Databases
 - Kubernetes
 - Logging Aggregator

DevOps

The infrastructure and pipelines more specific to the application. These could be logical structures or these could be the base, but overall it should follow a basic standard for the organization as much as possible.

- Kubernetes Region
- CodeBuild Deployment
- Secret Configuration
- Database/Schema Deployment
- Pod Configuration
- Storage Configuration

Development

The application that is being developed/maintained/deployed. What are the components that is need and how are they deployed.

- Containers with compiled code
- Zip files with compiled code
- DB Schema files (create/update)
- Environment Variable Definitions (secrets and .env)

Cloud Cross Reference

The Same but different

Feature	Google Cloud (GCP)	AWS	Microsoft Azure
Org Structure	Project (Top-level container)	AWS Account	Subscription / Resource Group
Compute (VMs)	Compute Engine	EC2 (Elastic Compute Cloud)	Azure Virtual Machines
Kubernetes	GKE (Google Kubernetes Engine)	EKS (Elastic Kubernetes Service)	AKS (Azure Kubernetes Service)
Object Storage	Cloud Storage (Buckets)	S3 (Simple Storage Service)	Blob Storage
Virtual Network	VPC (Global by default)	VPC (Regional by default)	Virtual Network (VNet)
Load Balancing	Cloud Load Balancing	ELB (ALB / NLB / CLB)	Azure Load Balancer / App Gateway
Serverless (Code)	Cloud Functions / Cloud Run	Lambda	Azure Functions
Relational DB	Cloud SQL / Spanner	RDS / Aurora	Azure SQL Database
Identity / Auth	Cloud IAM	IAM	Microsoft Entra ID (formerly AD)
Private Connect	Cloud Interconnect	Direct Connect	ExpressRoute

KurvPay

Task

What services does your application need today.

- What are all of the components
- What does it take to run your application/component
 - Compute
 - Storage
 - Network
 - Memory
- Don't worry about translating it to AWS yet.



KurvPay

What services does your application need today.

- Discussion about this item. What are the components, what do they need to run today.
 - Compute
 - IIS - Windows - .framework (4.6+)
 - IIS - Windows - .NET Core
 - sftp server
 - active directory authentication (intranet vs external (username / password))
 - Storage
 - SQL Server (2016 → 2022) (8c, 128GB, 9TB) - store proc permissions in ad
 - sftp volume
 - document storage volume (temp)?
 - laser fiche storage via API/UI
 - Network
 - API for Internal
 - Pull from External B2B (all internet)
 - API for external (all internet)
 - Memory
 - nothing beyond servers

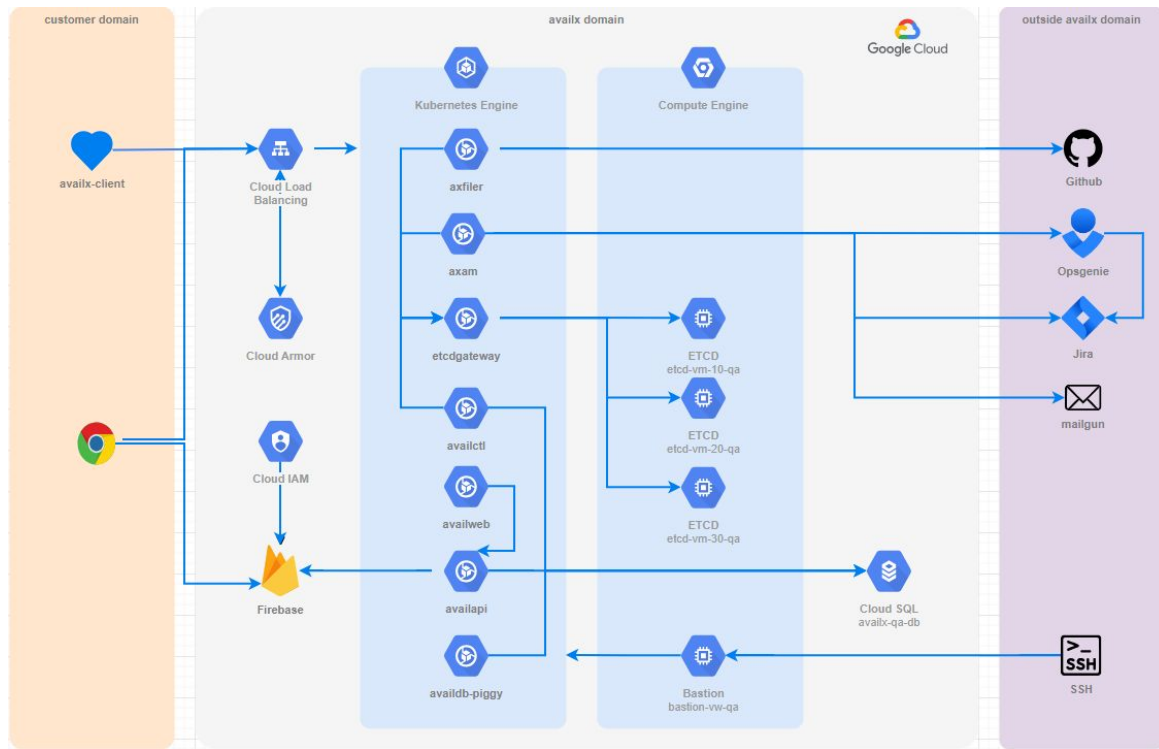


Infrastructure view

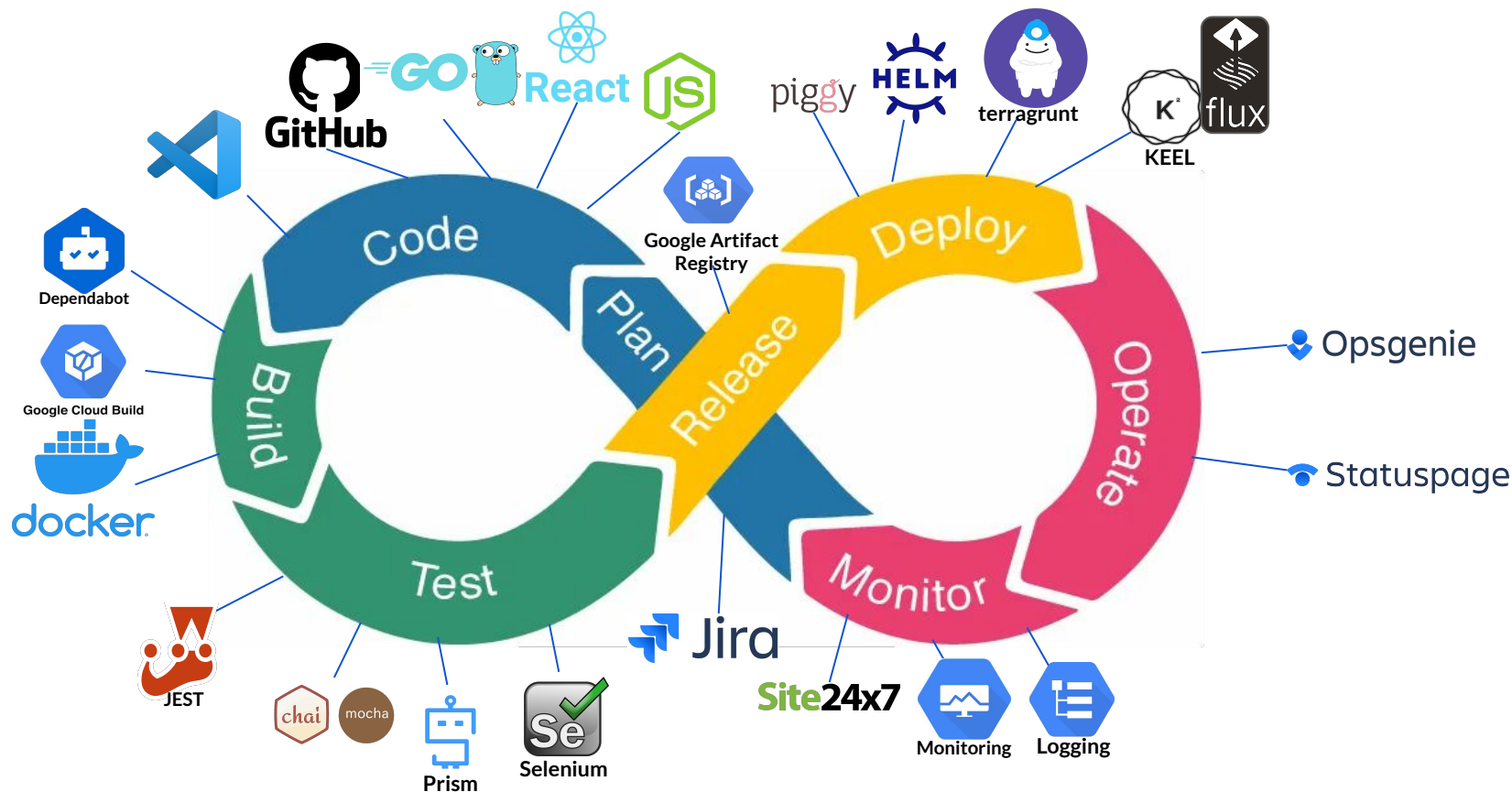
Modern Infrastructure

AvailX has a modern cloud native infrastructure and application implementation. Foundationally on Kubernetes, Containers and other managed services like Cloud SQL and Opsgenie.

This infrastructure allows for cloud security and scalability to be baked into AvailX from the start and means we can scale to any level required by our customers.



AvailX DevOps



AvailX Example

Basic, but not

Base Infrastructure

What are the base items required.

- GCP Project (limited "owner")
 - 1 project per app instance
 - 2 primary instances (QA/Prod)
- Rights differential
 - QA - Devs have read to everything (including secrets), data write to Database.
 - Prod - Devs have log access read logs, read access to database, write access is handled via "break glass"
- Initially all manual, roles and security profiles automated as application matured

DevOps

The infrastructure and pipelines more specific to the application.

- GKE - Managed ***
- Cloud SQL for PostgreSQL Managed ***
- VM cluster for ETCD
- Classic Load Balancing - Managed to the GKE Cluster
- CloudBuild - Pipelines
- Cloud Storage (bucket)
- Artifact Registry (Container Images)

Development

The application that is being developed / maintained / deployed. Basically Git

Application:

- Repos:
 - axweb (hosted)
 - availx (client)
- Agent:
 - availx (go)
- Containers:
 - Web Server (nginx/reactjs)
 - Web Services (Go/node)
 - Database Scripts for Schema

Infrastructure:

- Repos
 - axinfra - Initial
 - axdeploy - later
- Manual Steps
 - Action Plans (scripts)
 - GKE configuration yaml

KurvPay

What do you think these services would map to.

For the list of items Application and Infrastructure required, how would you map those into AWS



Cloud Cross Reference

The Same but different

Feature	AWS
Org Structure	AWS Account
Compute (VMs)	EC2 (Elastic Compute Cloud)
Kubernetes	EKS (Elastic Kubernetes Service)
Object Storage	S3 (Simple Storage Service)
Virtual Network	VPC (Regional by default)
Load Balancing	ELB (ALB / NLB / CLB)
Serverless (Code)	Lambda
Relational DB	RDS / Aurora
Identity / Auth	IAM
Private Connect	Direct Connect

- Compute
 - IIS - Windows - .framework (4.6+)
 - Ec2 - IIS VM
 - IIS - Windows - .NET Core
 - dockerfile update for linux container
 - Ec2 IIS vm - if windows/ad required/volume
 - sftp server
 - Ec2 or a fully managed service
 - active directory authentication (intranet vs external (username / password)
 - Ec2 instance.
 - What is your Container Runtime
 - EKS
- Storage
 - SQL Server (2016 → 2022) (8c, 128GB, 9TB) - stored proc permissions in ad
 - EC2 Cluster AG/ RDS for SQL Server
 - sftp volume
 - traditional volume or S3
 - document storage volume (temp)?
 - traditional storage (possible via map)
- Network
 - API for Internal
 - laserfiche storage via API/UI
 - VPN connection - DNS redirect
 - Pull from External B2B (all internet)
 - limiting container internet access
 - API for external (all internet)
 - Some source ip restriction, some "wide open", Generally limited via ELB
- Memory
 - nothing beyond servers (no redis cache or memstore)

KurvPay

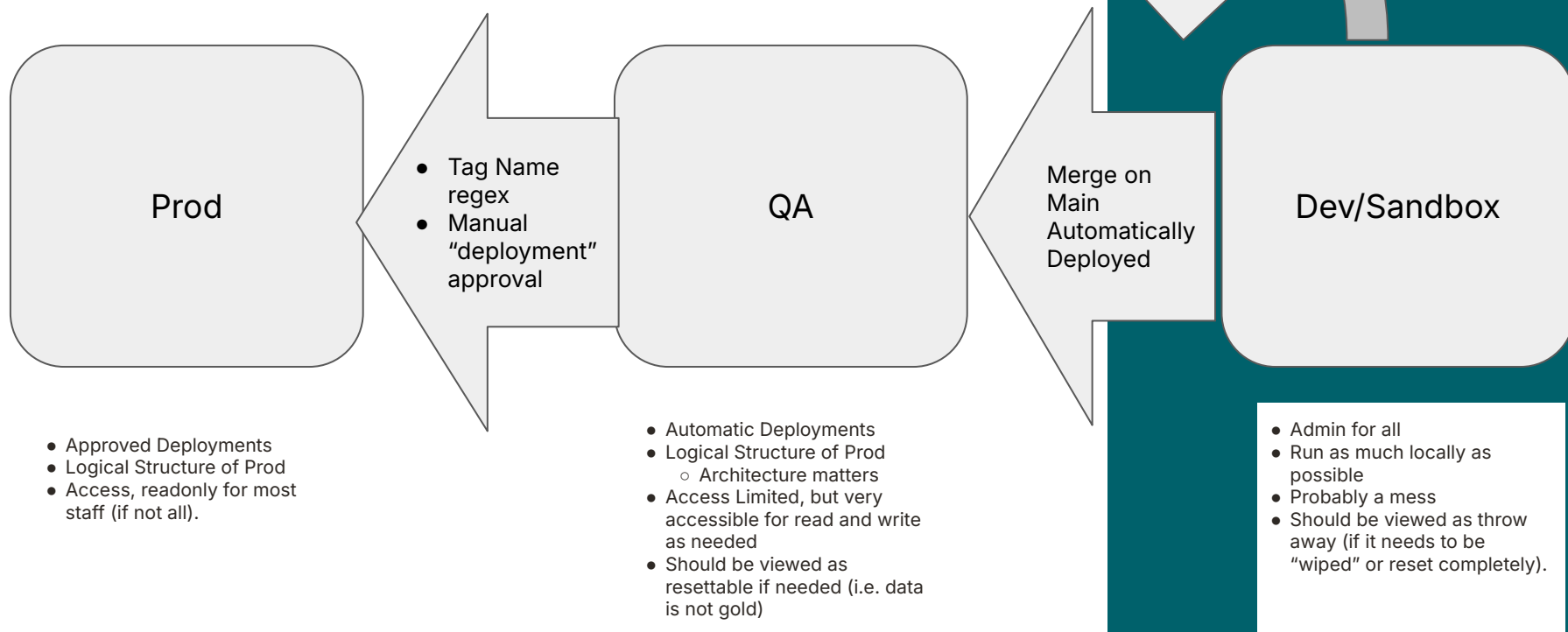
Review

Pythian



AvailX Example

Deployment - Controls



AvailX Example (IaC Tool)

We chose Terraform/Terragrunt mainly because of expertise and support. At Pythian Terraform knowledge is well known and supported (among others) so due to these reasons we chose it even though we did need to “backed it in” to an existing environment.

Terraform/Terragrunt (OpenTofu)

Declarative Yaml

You define what you want in Yaml files that are “merged”

The tool figures out what needs to be done by comparing the final output to a “known state”.

Pros:

- Readability - Infrastructure
- Most common Tool
- State Management - System will “know” if something changed.

Cons:

- Customization can be complex (loops/if)
- State needs to stay in check
- Difficult to overlay existing

Pulumi

Declarative Code

You write deployments more like code in Python, Javascript, Go or Java which are compared to the state

The tool figures out what needs to be done by comparing the final output to a “known state”.

Pros:

- Readability - Developer
- Full IDE support
- Full “language” support

Cons:

- Spaghetti Code
- Heavier runtime, slower execution
- Not as well supported as terraform

Pythian Services Inc. | Confidential | 18

Cloud Native IaC (CloudFormation)

Declarative Yaml/Code

There are choices here which can be either option.

The tool figures out what needs to be done by comparing the final output to a “known state”.

Pros:

- No infrastructure needs
- Day-One support for APIs
- Native Integration

Cons:

- AWS Lockin
- Verbose and Slower than Terraform
- Customization Complexity (loops/if)

AvailX Example (Package Manager Tool)

Helm

Declarative Go Templates

You define what the configuration looks like in a template and then “merges” changes per environment.

Helm defines the structure, FluxCD deploys and watches for changes.

Pros:

- Industry Standard
- Release/Rollback management
- Dependency management

Cons:

- Templates can be difficult to read
- Often difficult to debug (what is really in the yaml)
- Complex

We chose Helm mainly because of support. There are many known helm templates that come with various packages in the container world. At the same time it gave us automated backout capabilities during a release which we didn't have previously.

FluxCD detect changes and creates the “automation loop”.

Kustomize

Declarative Yaml w Overlay

You write Yaml configuration files and then overlay required changes per environment.

Kustomize structures are the “default” for Kubernetes.

Pros:

- No new language, just default kubernetes
- Built in - no additional components
- Predicable - WYSIWYG

Cons:

- No real packaging/version tracking
- Limited abilities (loops/if)
- No real template support

FluxCD

GitOps Tool

Watches the configuration of an environment as well as other components (like your container registry) and will update your github configuration files to apply the update.

This can be fully automated (when it detects the change it will deploy the containers) or require an approval (via a PR) such that the change is not applied until it is merged to main.

AvailX Example

Basic, but not

Deploy Infrastructure

Terraform/Terragrunt

Control of our infrastructure and infrastructure configuration

- Compute Package deployment
- GKE nodes
- CloudSQL Instances
- Load Balancing
- Cloud Armor
- Network Configuration
- Cloud Functions
- Composer (Airflow)
- Cloud Storage (buckets)

Manual : Secrets

Deploy Configuration

FluxCD

GitOps Agent

- Monitors for Changes in Environment (new container images)
- Deploys defined items when applicable (automatically or controlled via PR)
- Runs Helm Charts

External Secrets

- Syncs Secrets to GKE and restarted container affected

Deploy Application

Helm

Charts control the pod/container configuration on Kubernetes

- Controls "Pre Containers"
- Controls Environment Variables

CloudBuild

Receives the Event (hook) to build the packages necessary for future steps

AvailX Example (Custom Stuff)

While Terraform, Helm and FluxCD are our key technologies unfortunately they can't do everything (at least when we needed it).

- external secrets - utility to synchronize a secret from Google Cloud Secrets to GKE
- bash - various utilities to do things like package an item and put it somewhere for the next component.
 - a good example of this was how to version and update a Google Cloud Function properly
- Pub/Sub Queues - Various items needed to be updated when a build event happened at an application level. Custom subscription code was build to "see" these events and update various component properties (agent version and checksums)
- Packer - although we don't use many we finally implemented packer for VM template management.

This is just to say that while you don't want to reinvent the wheel for things that exist, you will have needs that are more custom to your applications requirements.

KurvPay

Discussion on Tooling

What will make sense at KurvPay?

- What expertise already exists in the Organization?
 - Currently On-Premise is manual
 - visual studio for code
 - redgate for schema diff and deploy
 - AWS
 - CLI "custom" command no repo
 - no app deployment yet
- Are there any tools that are in use, are they working for you?
 - All new



AvailX Example

Deployment - Controls

Prod/DR

- Approved Deployments
- Logical Structure of Prod
- Access, readonly for most staff (if not all).
- Same Automation as other environments

UAT

- Scheduled Deployments
- Logical Structure of Prod
 - As close as possible costs are likely to be an issue though
- Access, the same as Prod or very very very close.

QA

- Automatic Deployments
- Logical Structure of Prod
 - Architecture matters
- Access Limited, but very accessible for read and write as needed
- Should be viewed as resettable if needed (i.e. data is not gold)

Dev/Sandbox

- Admin for all
- Run as much locally as possible
- Probably a mess
- Should be viewed as throw away (if it needs to be "wiped" or reset completely).

Focus and Time

What should we be automating

Focus

What is really important

- What is eating a lot of time for a lot of people.
- What is eating a lot of time for people who are unavailable.
- What processes are "causing grief".
- Implement a process from "End to End" such that you can leave it behind.
- What may be timely due to other changes.
- Who is really responsible for what?
- Do not spend a lot of time focusing on exceptions and "what about-ism".
- You will need to create hacks at times to get around constraints.

Time

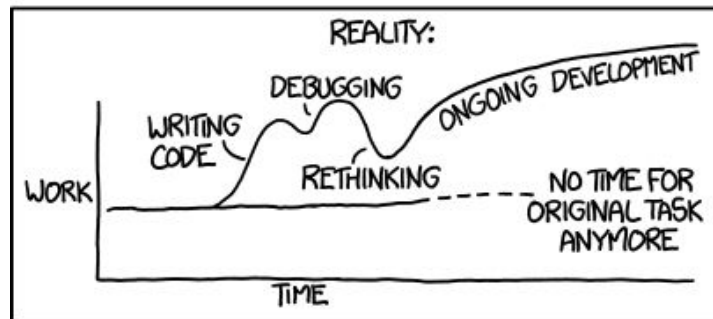
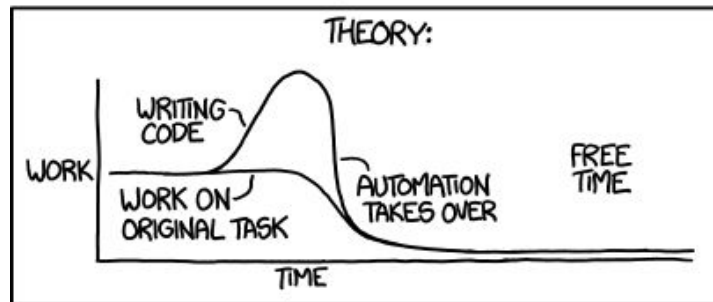
Rome was not built in a day.

- Be realistic about how much time process change takes
- Be realistic about how much time culture shift takes
- Be realistic about how much time it will take to learn a new technology(ies).
- Understand that not everything will work the first time.
- Give time to help fix/iterate on the process.
- You will create "hacks" at times for various reasons. Understand how to find them later to fix them vs lamenting they exist.

2 of my favorites

XKCD

"I SPEND A LOT OF TIME ON THIS TASK.
I SHOULD WRITE A PROGRAM AUTOMATING IT!"

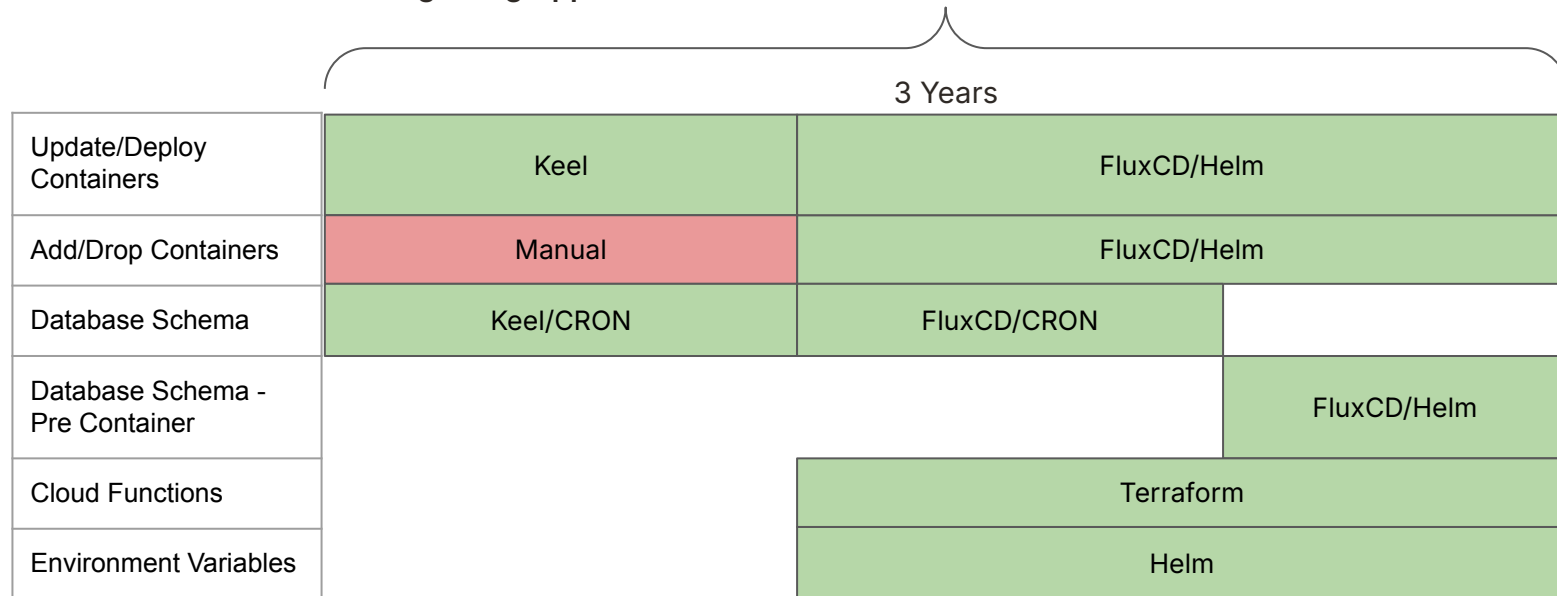


HOW LONG CAN YOU WORK ON MAKING A ROUTINE TASK MORE
EFFICIENT BEFORE YOU'RE SPENDING MORE TIME THAN YOU SAVE?
(ACROSS FIVE YEARS)

		HOW OFTEN YOU DO THE TASK					
		50/DAY	5/DAY	DAILY	WEEKLY	MONTHLY	YEARLY
HOW MUCH TIME YOU SHAVE OFF	1 SECOND	1 DAY	2 HOURS	30 MINUTES	4 MINUTES	1 MINUTE	5 SECONDS
	5 SECONDS	5 DAYS	12 HOURS	2 HOURS	21 MINUTES	5 MINUTES	25 SECONDS
	30 SECONDS	4 WEEKS	3 DAYS	12 HOURS	2 HOURS	30 MINUTES	2 MINUTES
	1 MINUTE	8 WEEKS	6 DAYS	1 DAY	4 HOURS	1 HOUR	5 MINUTES
	5 MINUTES	9 MONTHS	4 WEEKS	6 DAYS	21 HOURS	5 HOURS	25 MINUTES
	30 MINUTES		6 MONTHS	5 WEEKS	5 DAYS	1 DAY	2 HOURS
	1 HOUR		10 MONTHS	2 MONTHS	10 DAYS	2 DAYS	5 HOURS
	6 HOURS				2 MONTHS	2 WEEKS	1 DAY
	1 DAY					8 WEEKS	5 DAYS

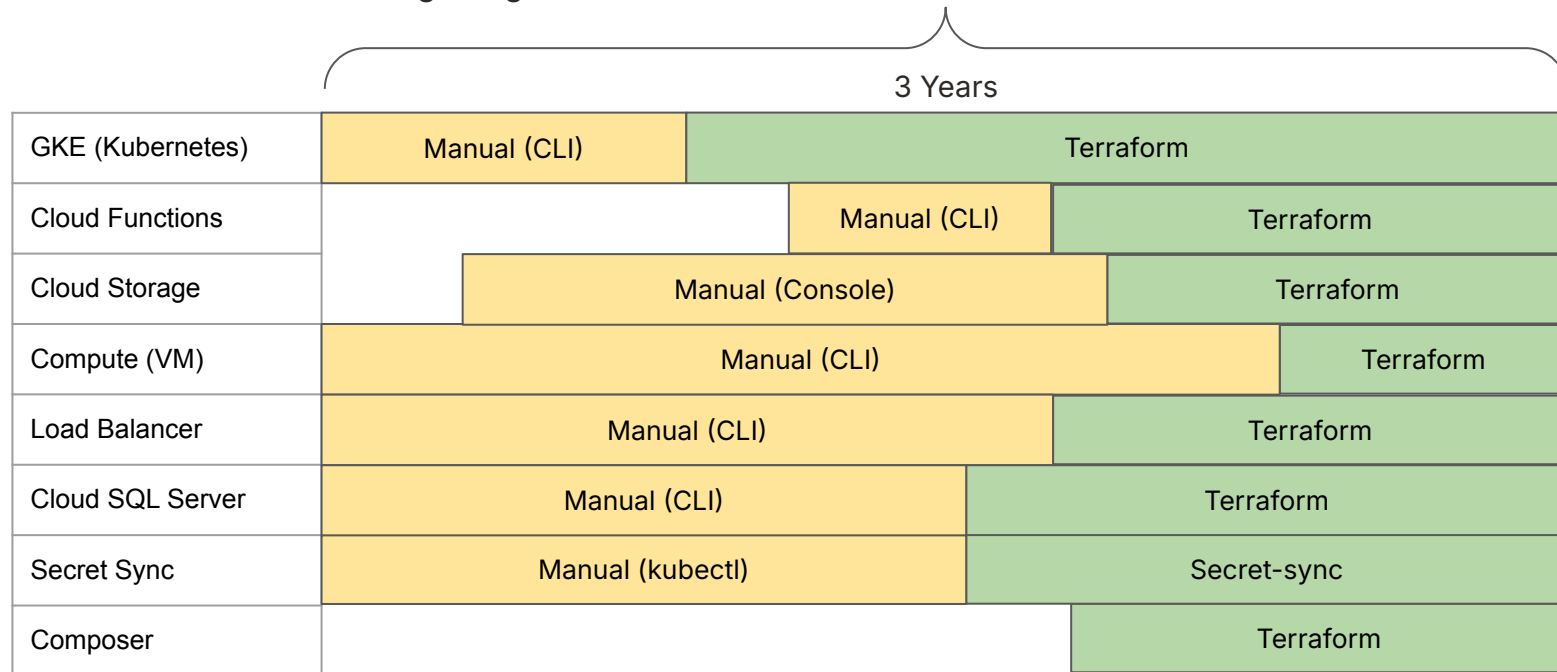
AvailX Implementation

We didn't have it all at the beginning Application



AvailX Implementation

We didn't have it all at the beginning Infrastructure



Process

The tool dictates the workflow



Comments to Think about

You will need to be able to separate the big boxes from the smaller detailed processes.

- Where does the big box begin and end
- How do you know the process was successful/failed
- Where are the boundaries the need to involve more than 1 group/role
- How do you enable a group to be self sufficient (at least most of the time).

Code Deployment Pipeline

Basic Example of "Update Code for Container" for 1 environment (say QA)

Devops

Shared

Development

What is Devops' accountability?

To have the working code deployed to the correct environment when it is supposed to be deployed.

What does Devops need for this?

- Working compiled code
- Environment Variables necessary for a given environment.
- Secrets necessary for a given environment
- Infrastructure available for deployment of code

What does Development need from Devops?

- Source Code Repository
- Container Registry
- Build Tools available

What does Devops from Development?

- Environment Variables List
 - What are secrets?
 - What are the values (if applicable)
- Compiled code in the Container Registry
- What is "supposed" to be deployed.

What is Development's accountability?

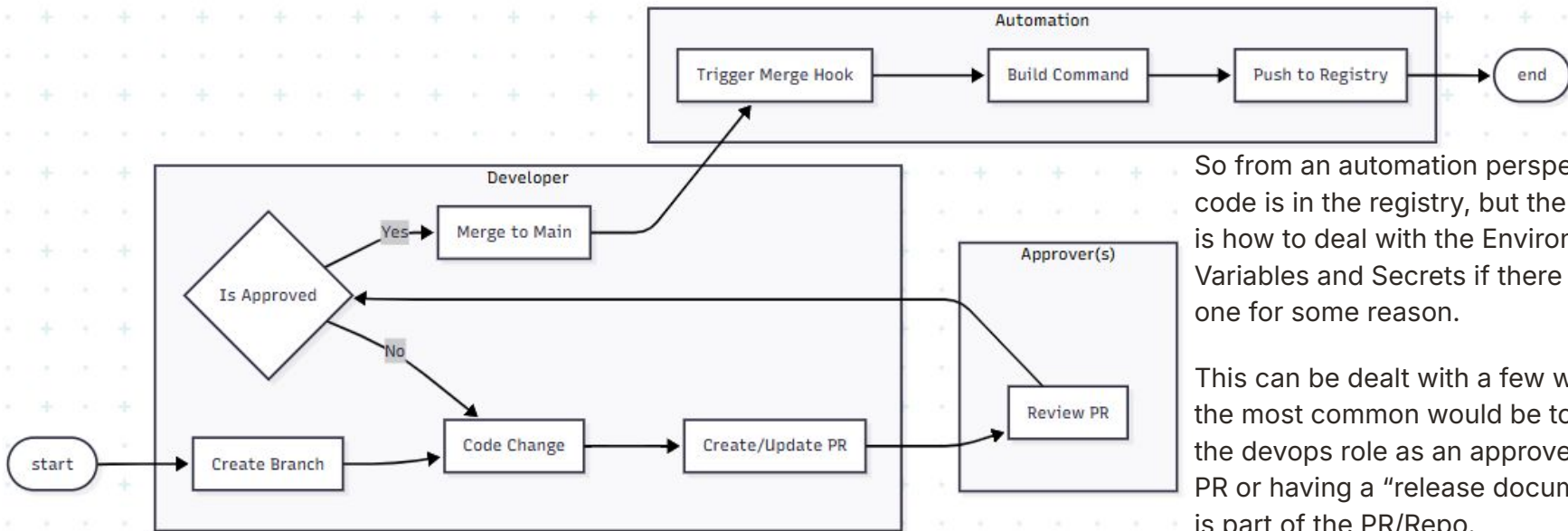
To have a working compiled code ready for deployment.

What does Development need for this?

- Source Code Repository
- PR approval process
- A way to build the code
- A place to put the code ready for deployment

Detailed Development Process

Detailed Process



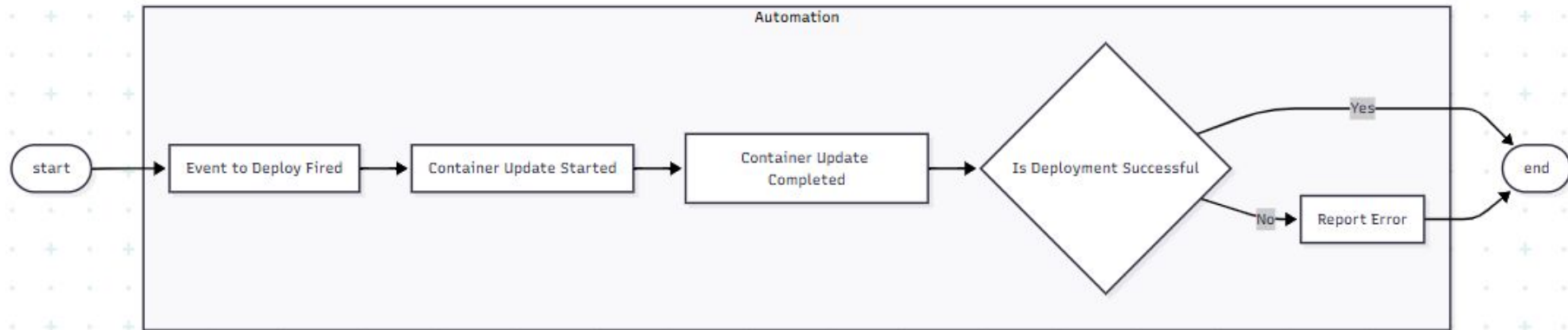
So from an automation perspective the code is in the registry, but the question is how to deal with the Environment Variables and Secrets if there is a new one for some reason.

This can be dealt with a few ways, but the most common would be to include the devops role as an approver on the PR or having a "release document" that is part of the PR/Repo.

Good Error logging will make this easier

Detailed Deployment Process

Detailed Process



From an automation perspective the code is in the registry, but the question is how to deal with the Environment Variables and Secrets if there is a new one for some reason.

This will depend on the process and the tools. You can automate it as part of the deployment process, or have it as a manual step before the Event to Deploy is fired.

Monitoring is a large part of this to know when something fails, good error logging will matter.

What Processes do you have today

Thinking about the processes you have today

- What would work with few changes?
 - Code build and deploy to an environment?
- What would require major changes?

- When looking at a “manual” process to automate remember:
 - These may not be a 1:1. 1 process manually may be 2 or more automated processes.
 - Things that take a long time to do in a manual process may take only seconds in automation, don’t complicate a process to make it “the same”.
- Focus on what is common first and will save time. Environment Variables may be a pain, but generally don’t change often. So not including them in the automation process at the beginning may be a reasonable option.

People

Pro Tip: Before you automate, use the "**Simplify, Eliminate, then Automate**" rule. If you can't explain the process clearly on a whiteboard without using the word "usually," it's not ready for a technical implementation.



Comments to Think about

Feature	Manual Process	Technical Process
Error Handling	Adaptive (Human logic)	Rigid (Pre-defined logic)
Speed	Variable/Slow	Constant/Fast
Consistency	Low (People have bad days)	High (Code is tireless)
Scalability	Hard (Requires more people)	Easy (Requires more compute)

Roles and Accountability

Roles need to be much more clearly defined:

- What is a role actually accountable for?
- Exactly what does that mean?
- Does everybody in that role agree/understand

While this is not technically an Org change some things will feel like it:

1. Organizations are implicitly optimized to avoid changing the status quo middle- and first-level manager and “specialist” positions & power structures.
2. As a corollary to (1), any change initiative will be reduced to redefining or overloading the new terminology to mean basically the same as status quo.
3. As a corollary to (1), any change initiative will be derided as “purist”, “theoretical”, “revolutionary”, “religion”, and “needing pragmatic customization for local concerns” — which deflects from addressing weaknesses and manager/specialist status quo.
4. As a corollary to (1), if after changing the change some managers and single-specialists are still displaced, they become “coaches/trainers” for the change, frequently reinforcing (2) and (3), and creating the false impression ‘the change has been done’, deluding senior management and future change attempts, after which they become industry consultants.
5. (in large established orgs) Culture follows structure. And in tiny young orgs, structure follows culture.

https://www.craiglarman.com/wiki/index.php?title=Larman%27s_Laws_of_Organizational_Behavior

Roles and Accountability

Much of this depends on your current Staffing and Mind set but common issues are:

Do'er to Approver: Automation tends to remove certain types of tasks and add others. Often this means that somebody that used to "review and run" a script, now "reviews and approves" a script that is run by some "magic" automation at some unspecified time. This can be very disturbing for some staff.

What is my job: Automation tends to change the tasks at hand. You may start to force scripts/configuration through the pipeline such that "first time" items can take longer. It is called "Infrastructure as Code" for a reason. Infrastructure guys maybe using source control for the first time, what used to be running a command for configuration is now "writing a script". This can make people feel like there job is completely different even if it is basically the same.

Roles and Accountability

Much of this depends on your current Staffing and Mind set but common issues are:

Automate what I don't like: Often people will try to automate tasks they dislike doing. This may be a large task, but is only done 1 time per year. An inordinate amount of time can be spent trying to automate things that don't really matter or would be very far down the list of things to do.

Heros are less important: Automation can (should) make the 3 am crazy call less likely/required. Some staff will feel this lose of importance and/or social status among their peers.

Mindset

Ultimately Devops and Automation at this level is part of the “Shift Left” and/or “Shift Right” mentality. How you get the people who are really going to “fix” the issues closer to problem without compromising security, compliance, user experience, etc.

At the end of the day you will be looking for how do you reward the behaviours you want and tamping down the behaviours you don't want.

- Make processes for change you want “easy”/frictionless. Make processes for legacy things “hard”/High friction.
- Reward sharing artifacts; often this is through things like source control over documentation
- Psychological Safety; make failure a mechanism to improve the process/system not to punish the wrongdoer.

Day 2 Setup

Is your setup working

- Can you access (<https://chatgpt.com>)
- Is VSCode Installed
(<https://code.visualstudio.com/download>)
- Is WSL Installed
(<https://learn.microsoft.com/en-us/windows/wsl/tutorials/wsl-vscode>)
- Is Docker Desktop Installed
(<https://www.docker.com/products/docker-desktop/>)
- Is Git Installed
- Github Repo - (<https://www.github.com>) You will need to be able to create an API Key for this repo.



Day 3 Setup

Is your setup working

- Items from Day 2
- aws cli installed (WSL is better)
(<https://dev.to/pu-lazydev/installing-aws-cli-v2-on-wsl-ubuntu-plg>)
- aws account with a good amount of create rights
(CodeBuild, App Runner, Elastic Container Registry, IAM
(roles), CloudFormation)

Questions & Next Steps



Thank you